Lecture 9: Prompt Engineering & RLHF

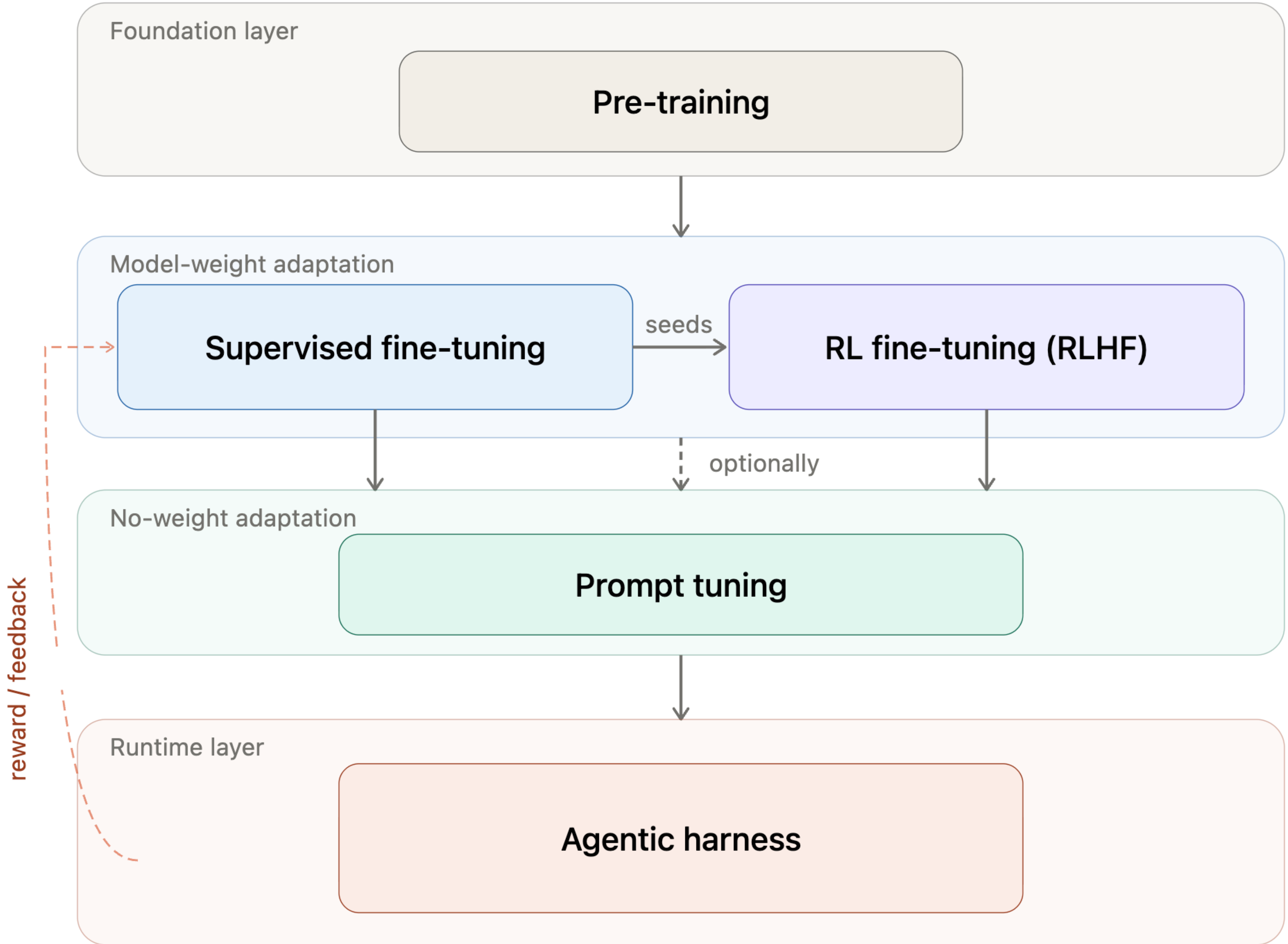
Kai-Wei Chang
CS @ UCLA
kw@kwchang.net

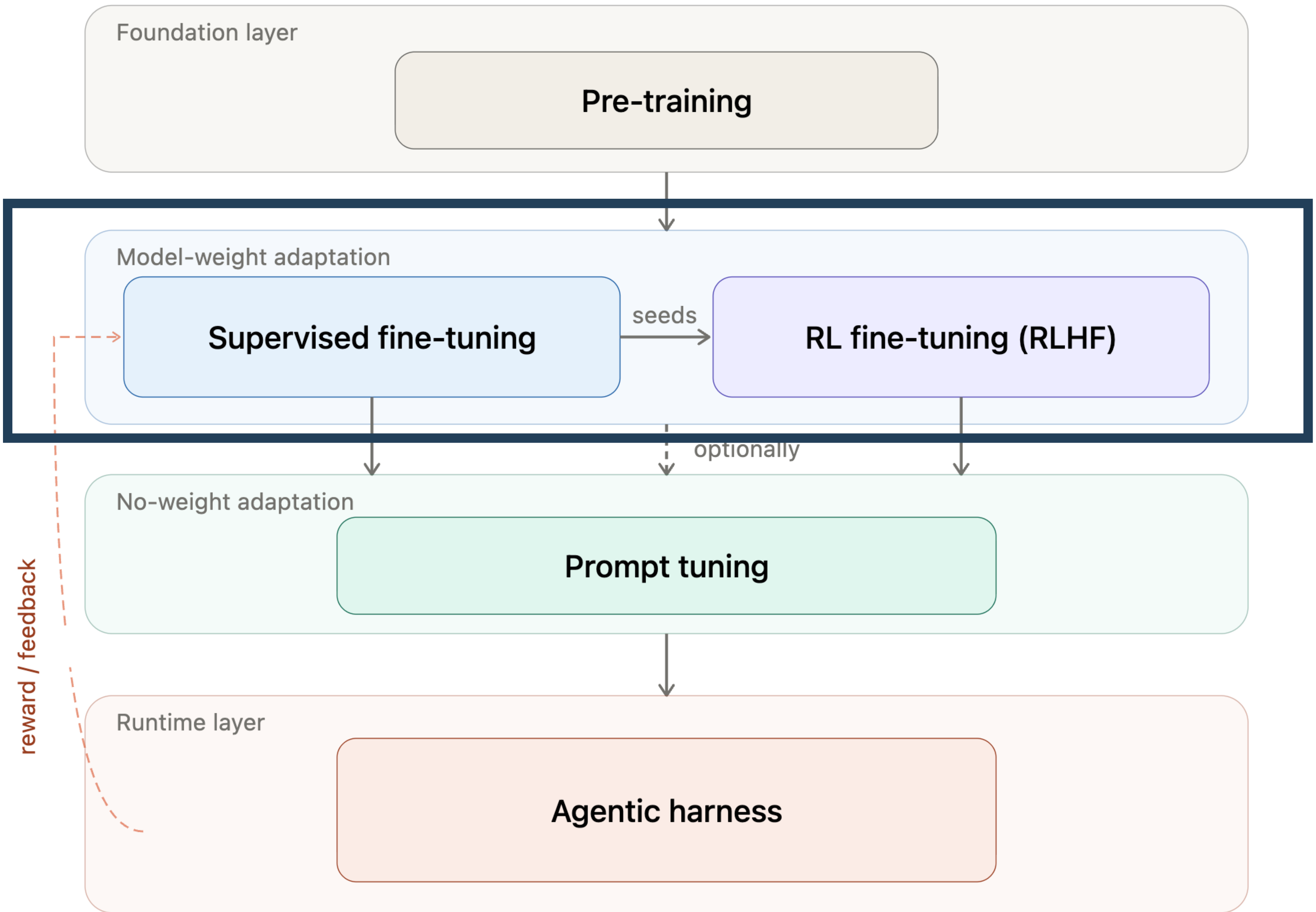
Credit: some slides are modified from Vivek Srikumar, Jason Eisner, Chris Manning

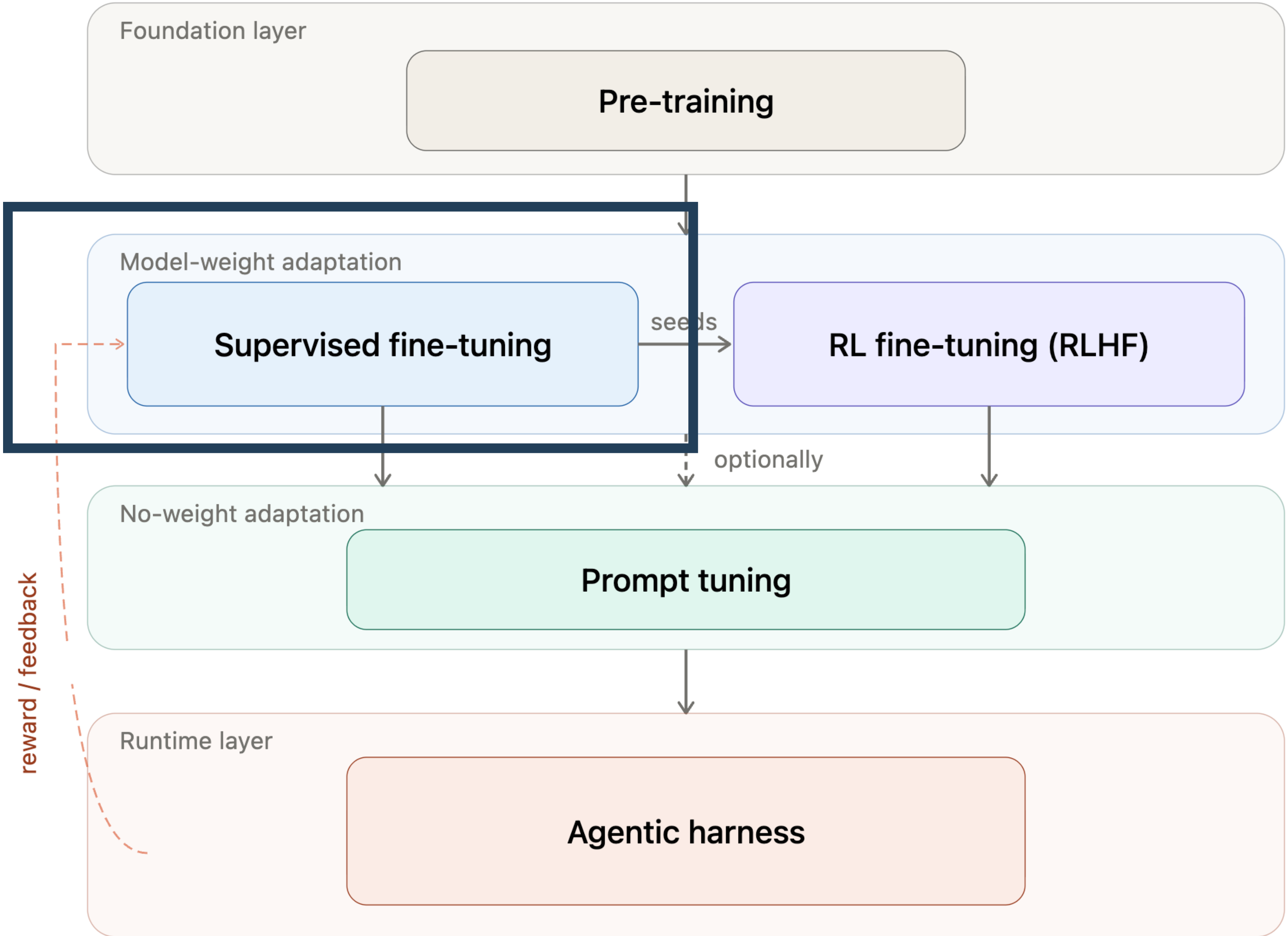
Announcement

- Midterm grade is released
  - Regrading request until next Monday
- Hw1 DEADLINE today
- Hw2 is released

https://chatgpt.com/


Foundation layer
Pre-training
Model-weight adaptation
Supervised fine-tuning
seeds
RL fine-tuning (RLHF)
optionally
No-weight adaptation
Prompt tuning
Runtime layer
Agentic harness
reward / feedback


Foundation layer
Pre-training
Model-weight adaptation
Supervised fine-tuning
seeds
RL fine-tuning (RLHF)
optionally
No-weight adaptation
Prompt tuning
reward / feedback
Runtime layer
Agentic harness


Foundation layer
Pre-training
Model-weight adaptation
Supervised fine-tuning
seeds
RL fine-tuning (RLHF)
optionally
No-weight adaptation
Prompt tuning
reward / feedback
Runtime layer
Agentic harness

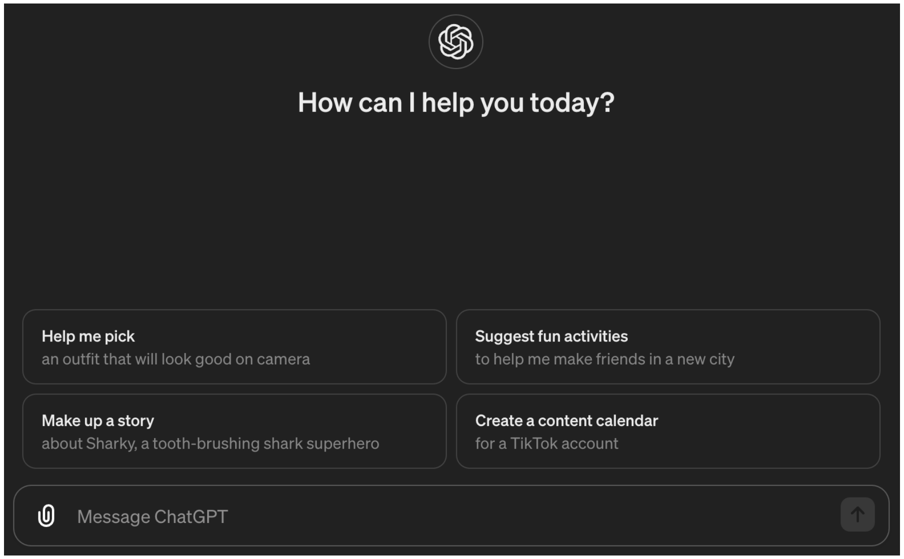
Instruction Fine-Tuning

Language modeling ≠ assisting users

How do we get from this

The student open their ____

to this?



UCLA ENGINEERING
Computer Science

Language modeling ≠ assisting users

Training language models to follow instructions with human feedback

Long Ouyang* Jeff Wu* Xu Jiang* Diogo Almeida* Carroll L. Wainwright*

Pamela Mishkin* Chong Zhang Sandhini Agarwal Katarina Slama Alex Ray

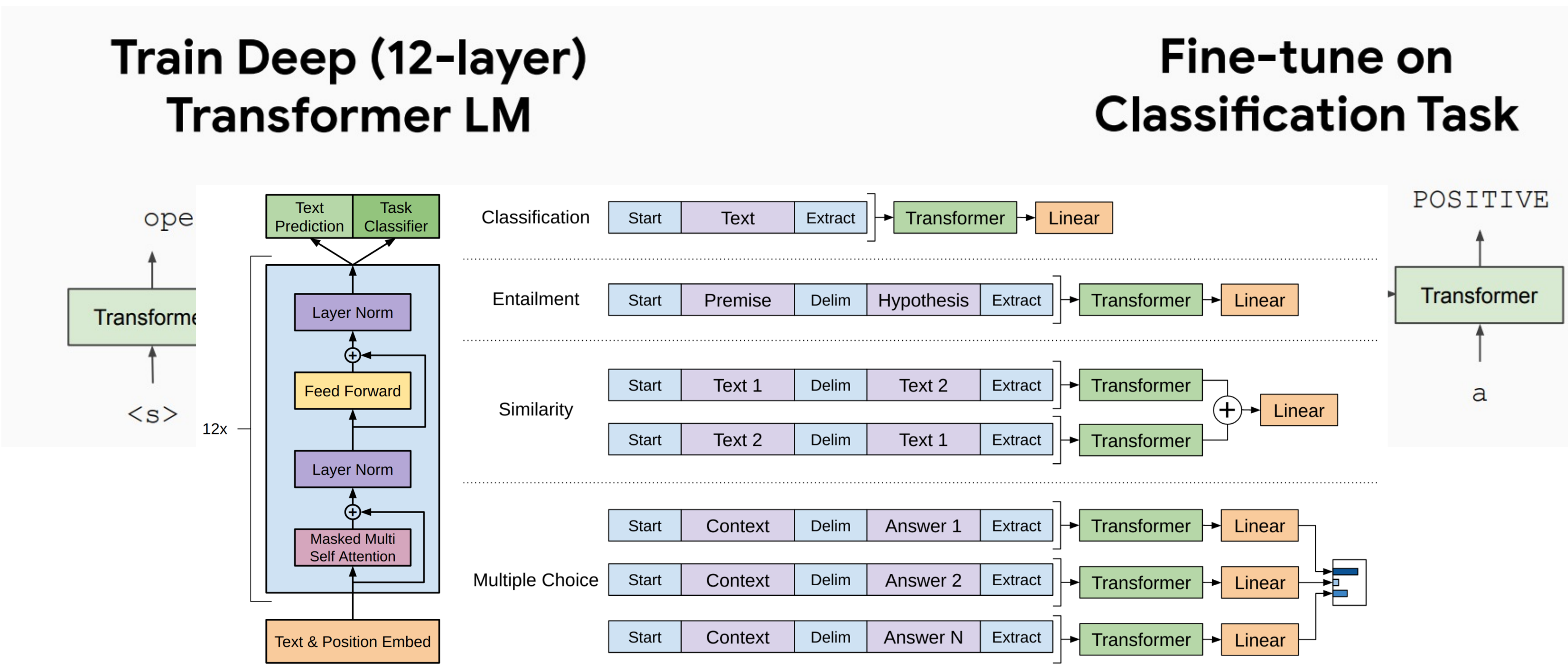
John Schulman Jacob Hilton Fraser Kelton Luke Miller Maddie Simens

Amanda Askell† Peter Welinder Paul Christiano*†

Jan Leike* Ryan Lowe*

OpenAI

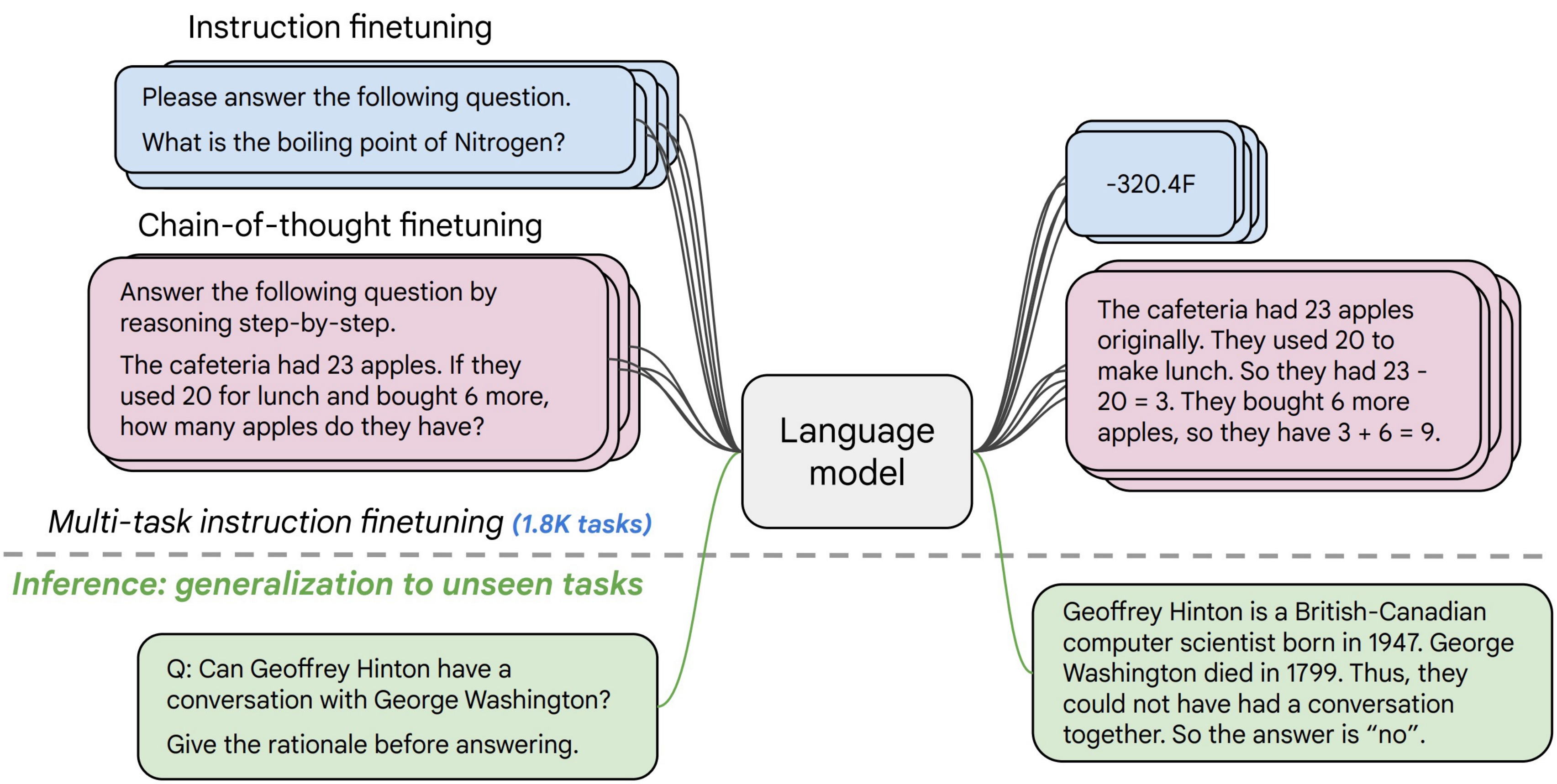
Recall GPT Pre-training and Fine-Tuning



UCLA ENGINEERING Computer Science

From Fine-Tuning To Instruction Fine-Tuning

- **Collect examples** of (instruction, output) pairs across many tasks and finetune an LM



- Evaluate on **unseen tasks**

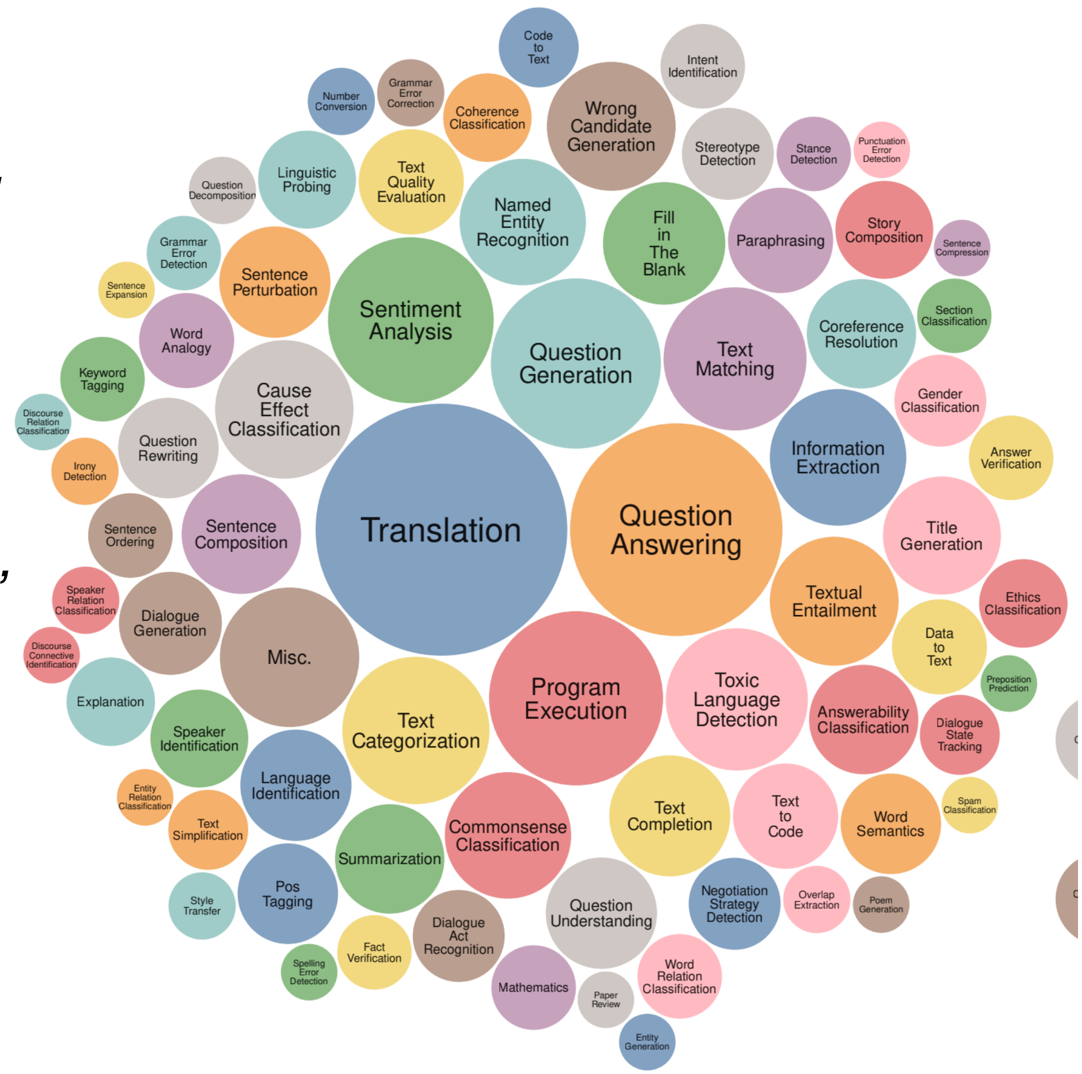
[FLAN-T5; Chung et al., 2022]

General Recipe For Instruction Fine-Tuning

- Find as **many** datasets as you can → there is a large number of "classical" NLP tasks, relatively diverse
- Convert them to text-to-text
- Mix-in instructions with examples
  - Multiple templates for each dataset
  - Fine-tuning for "next word prediction" task
- Split for train/test along tasks

Scale Up Instruction Fine-Tuning

- *As is usually the case, data + model scale is the key!*
- *For example, the Super-Natural-Instructions dataset contains over 1.6K tasks, 3M+ examples*
  - *Classification, sequence tagging, rewriting, translation, QA...*
  - *Aside: partly out of a class effort*



Super-NaturalInstructions[Wang et al., 2022]

UCLA ENGINEERING
Computer Science

Instruction Fine-Tuning Data Collection

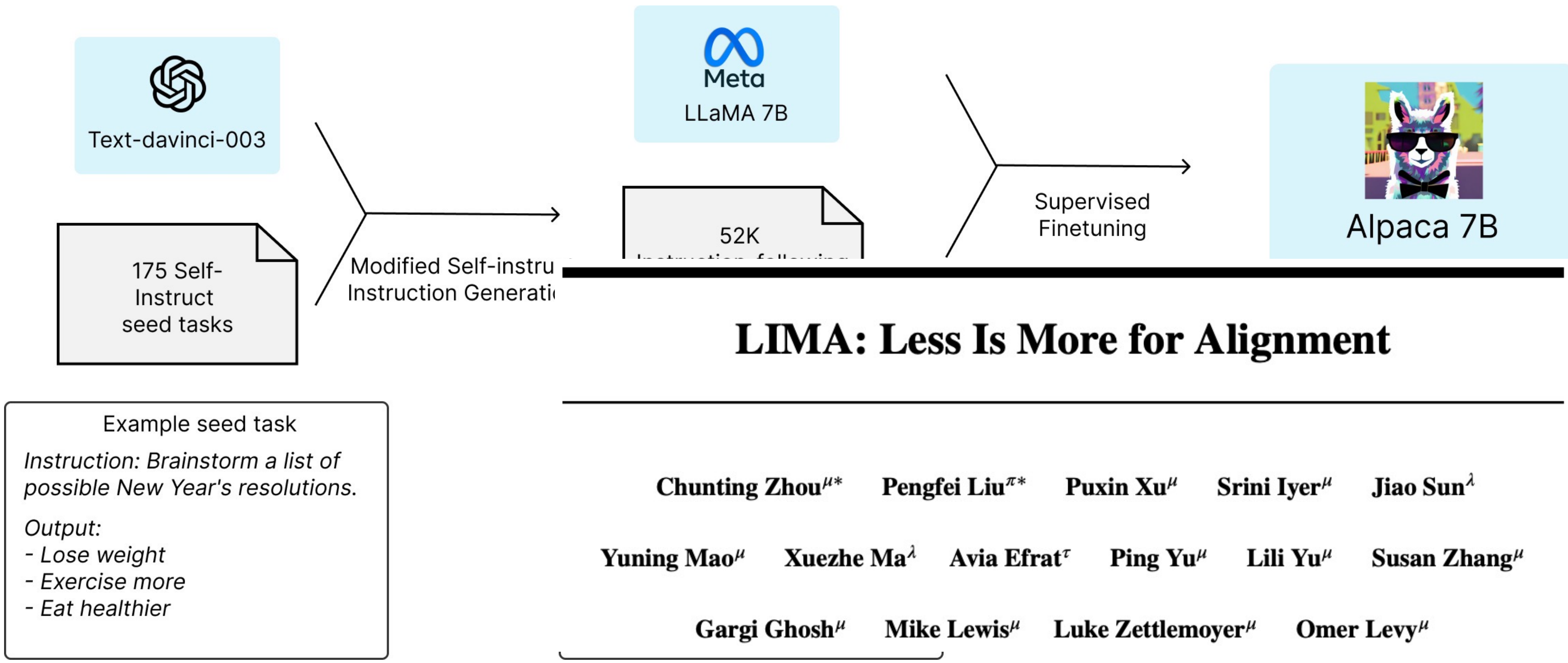
The Llama 2 (and closed-source LLMs) Recipe

- Emphasize data quality
- Hire third-party annotators
- Develop guidelines that match the desired model behavior
  - Llama 2 focus: helpfulness and safety
- Collect 27,540 examples
- Goal: good starting point for RLHF

More Instruction-Tuned Models

- Open-source instruction following models trained on synthetic data (from bigger LMs).

- You don't need many samples for instruction fine-tuning



LIMA: Less Is More for Alignment

Chunting Zhou[μ*] Pengfei Liu[π*] Puxin Xu[μ] Srini Iyer[μ] Jiao Sun[λ]
Yuning Mao[μ] Xuezhe Ma[λ] Avia Efrat[τ] Ping Yu[μ] Lili Yu[μ] Susan Zhang[μ]
Gargi Ghosh[μ] Mike Lewis[μ] Luke Zettlemoyer[μ] Omer Levy[μ]

Beyond Instruction Fine-Tuning

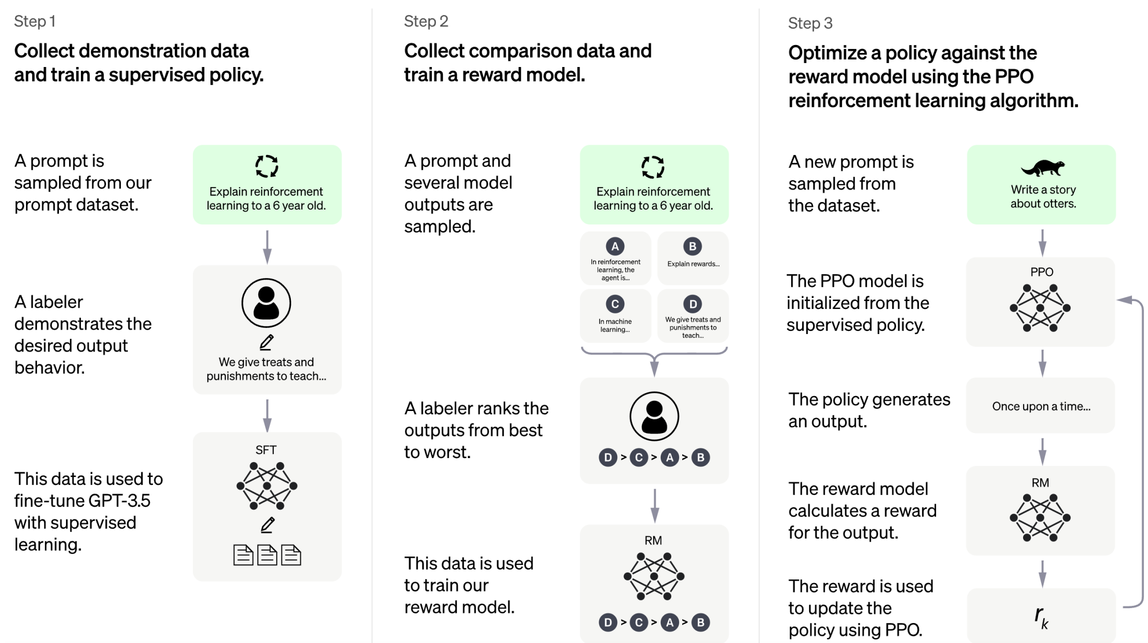
- SFT only reward **positive** examples, but there is no way to **penalize** *negative* outputs.
- Annotating data from scratch is very expensive.
- What if we can generate multiple outputs from model and obtain human feedback/preference? E.g., for a summarization task:

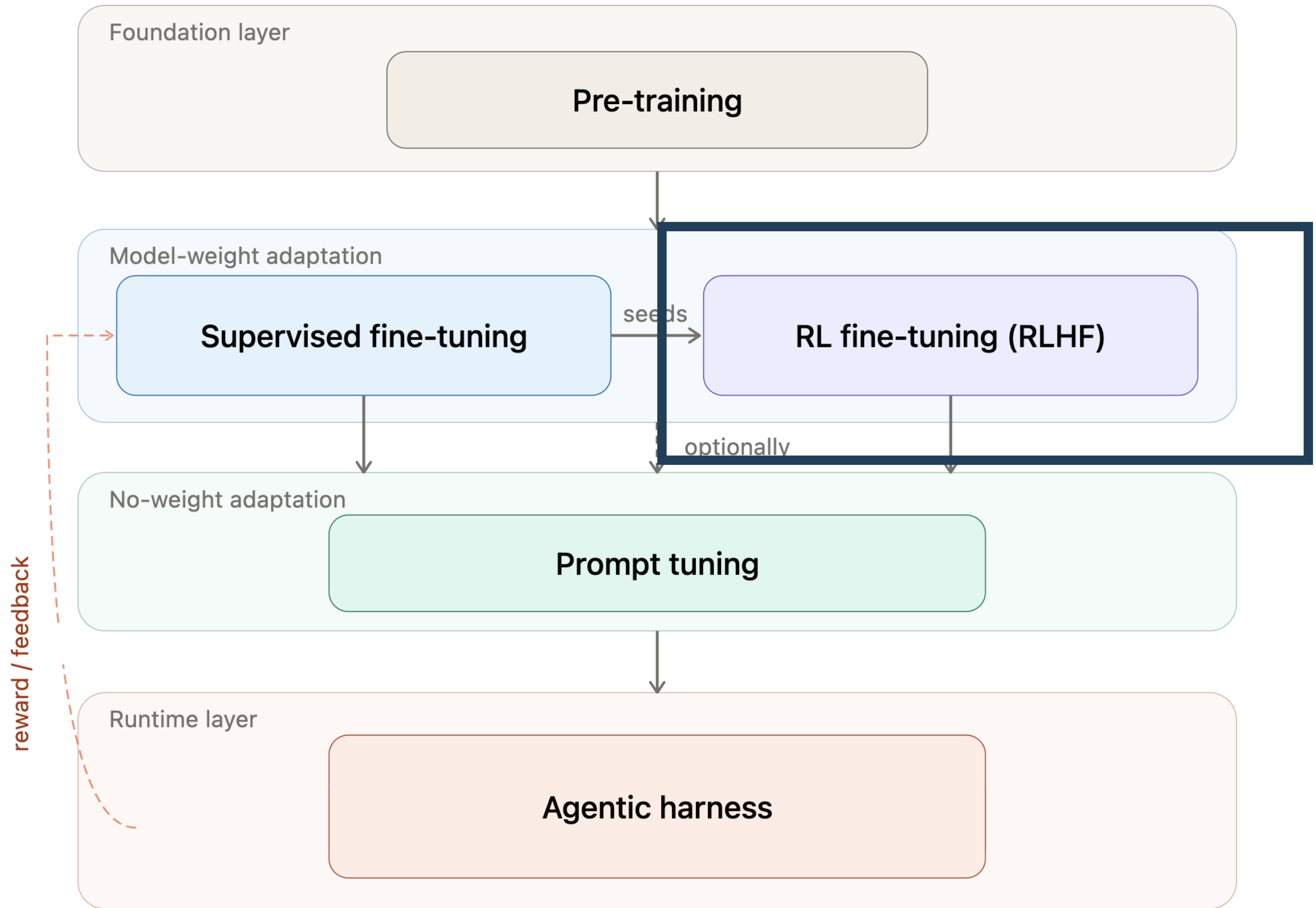
SAN FRANCISCO, California (CNN) -- A magnitude 4.2 earthquake shook the San Francisco ... overturn unstable objects.	An earthquake hit San Francisco. There was minor property damage, but no injuries.	The Bay Area has good weather but is prone to earthquakes and wildfires.
	s_1	s_2
	$R(s_1) = 8.0$	$R(s_2) = 1.2$

UCLA ENGINEERING Computer Science

Beyond Instruction Fine-Tuning

- Training LLMs: Post-training:
  - (Instructionally) Supervised Fine Tuning (SFT) and Reinforcement Learning from Human Feedback (RLHF)




Foundation layer
Pre-training
Model-weight adaptation
Supervised fine-tuning
seeds
RL fine-tuning (RLHF)
optionally
No-weight adaptation
Prompt tuning
Runtime layer
Agentic harness
reward / feedback

Reinforcement Learning from (Human) Feedback

UCLA ENGINEERING
Computer Science

Challenges of SFT

Challenges of SFT

- Writing a perfect answer is hard
- Many valid answer exist
  - We may penalize correct model outputs
- Annotations can be inconsistent

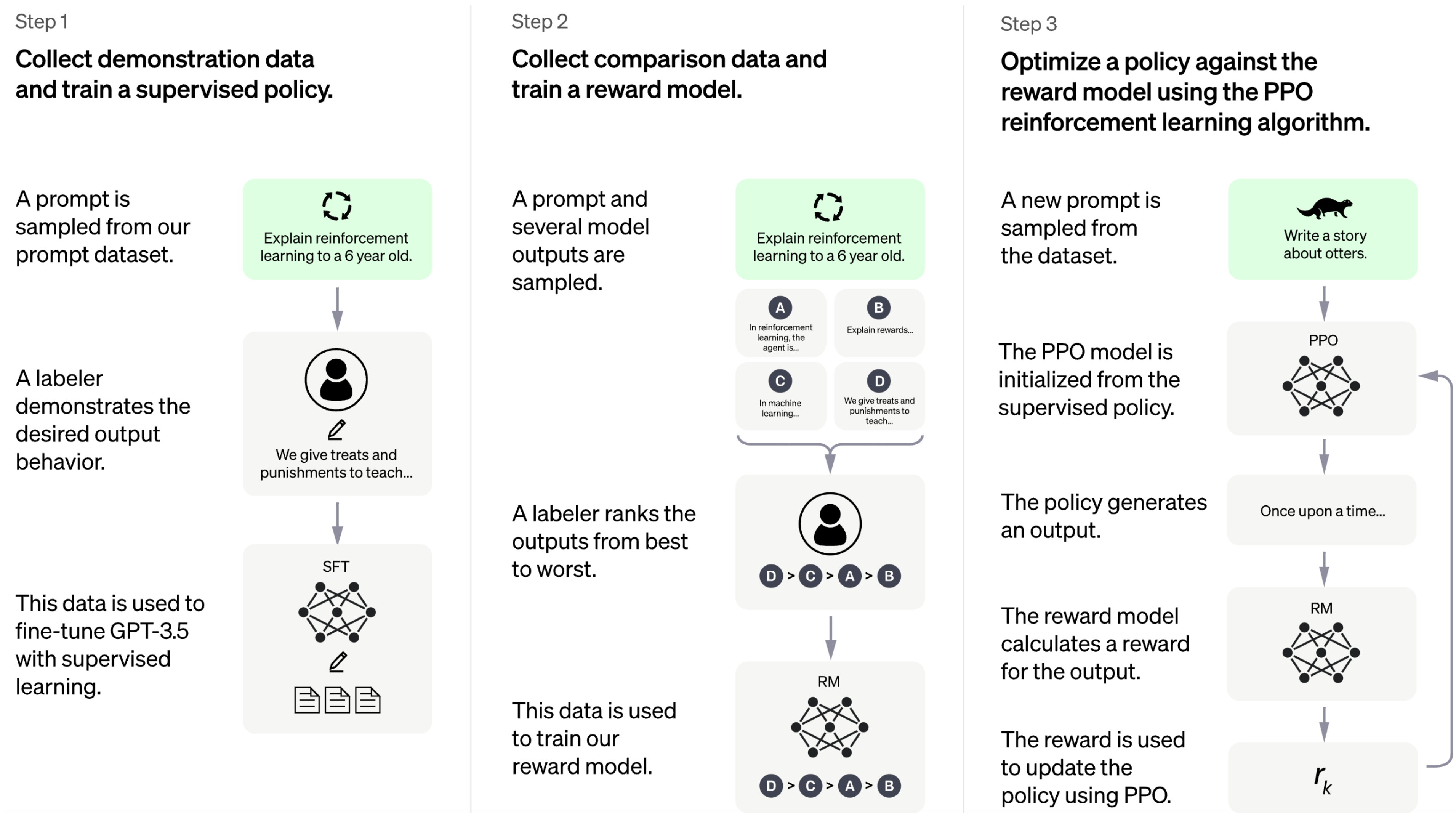
Data Annotation

- Score the helpfulness of the following response, 1-10

```
What are the steps for making a simple cake?

1.    Warm up the oven.
2.    Grease a cake pan.
3.    Blend dry ingredients in a bowl.
4.    Incorporate butter, milk, and vanilla.
5.    Mix in the eggs.
6.    Pour into the prepared pan.
7.    Bake until golden brown.
8.    Add frosting if desired.
```

Aligning LLMs



From [Ouyang et al., 2022]

RLHF – Data

- First step: need annotated data
  - To distinguish good responses from bad ones
- Need to annotate examples about whether they are good or bad
- No good automated metrics, because the text looks really good
- So, need to ask humans to evaluate

RLHF – Data Annotation

- Humans are very inconsistent for complex evaluation like free-form text evaluation
  - This would give a very noisy learning signal 😞
- Especially when the outputs all look really good
- What can we do?

RLHF – Human Preferences

- Which of these two responses is more helpful?

```
What are the steps for making a simple cake?

1.    Warm up the oven.
2.    Grease a cake pan.
3.    Blend dry ingredients in a bowl.
4.    Incorporate butter, milk, and vanilla.
5.    Mix in the eggs.
6.    Pour into the prepared pan.
7.    Bake until golden brown.
8.    Add frosting if desired.
```

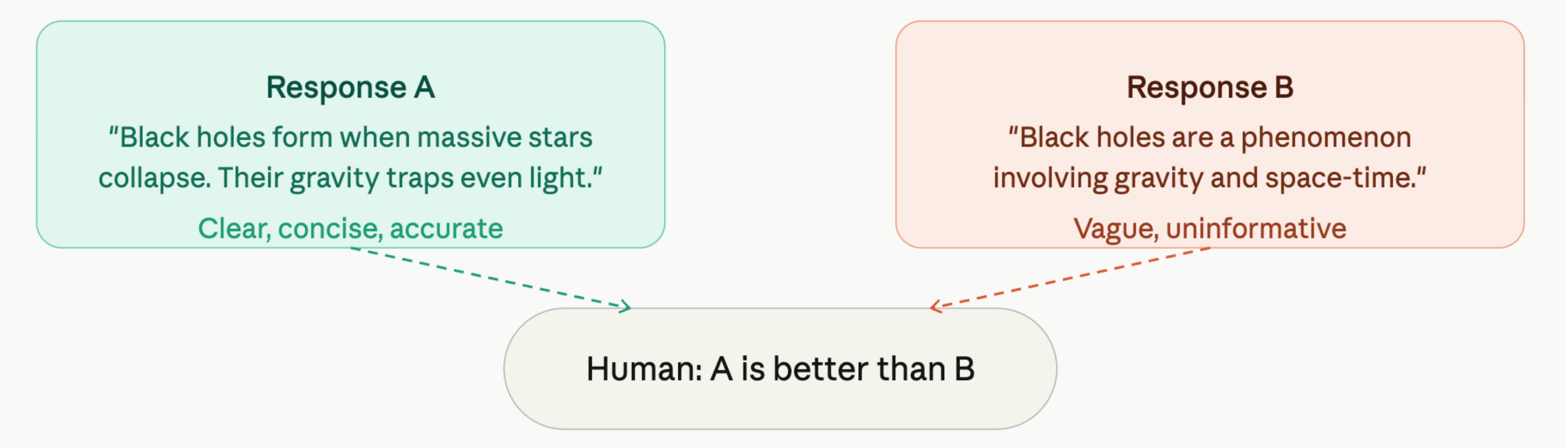
```
What are the steps for making a simple cake?

1.    Preheat oven to 350°F (175°C).
2.    Grease and flour a cake pan.
3.    In a bowl, combine 2 cups flour, 1.5 cups
      sugar, 3.5 tsp baking powder, and a pinch
      of salt.
4.    Add 1/2 cup butter, 1 cup milk, and 2 tsp
      vanilla; mix well.
5.    Beat in 3 eggs, one at a time.
6.    Pour batter into the pan.
7.    Bake for 30-35 minutes or until a toothpick
      comes out clean.
8.    Let cool, then frost or serve as desired.
```

UCLA ENGINEERING Computer Science

RLHF – Human Preferences

- Instead of evaluating a single example
  - Sample two outputs for the same input from the model
  - And choose a winner



UCLA ENGINEERING Computer Science

RLHF – Human Preferences

- Instead of evaluating a single example
  - Sample two outputs for the same input from the model
  - And choose a winner
- Formally, we get a dataset of inputs $\boldsymbol{x}^{\{i\}}$ paired with a winning output $\boldsymbol{y}_w^{(i)}$ and a losing output $\boldsymbol{y}_l^{(i)}$

$$\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}_w^{(i)}, \boldsymbol{y}_l^{(i)})\}_{i=1}^{N}$$

UCLA ENGINEERING Computer Science

RLHF -- Learning

- Now we have a dataset of inputs $\boldsymbol{x}^{\{i\}}$ paired with a winning output $\boldsymbol{y}_w^{(i)}$ and a losing output $\boldsymbol{y}_l^{(i)}$

$$\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}_w^{(i)}, \boldsymbol{y}_l^{(i)})\}_{i=1}^{N}$$

- We want to learn to generate outputs $\mathbf{y}^{(i)}$ given inputs $\mathbf{x}^{(i)}$
- How do we learn from this data?
  - Can we just pretend $\boldsymbol{y}_w^{(i)}$ are annotated outputs?
  - Do we just throw $\boldsymbol{y}_l^{(i)}$ away?

RLHF -- Learning

- Now we have a dataset of inputs $\boldsymbol{x}^{\{i\}}$ paired with a winning output $\boldsymbol{y}_w^{(i)}$ and a losing output $\boldsymbol{y}_l^{(i)}$

$$\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}_w^{(i)}, \boldsymbol{y}_l^{(i)})\}_{i=1}^{N}$$

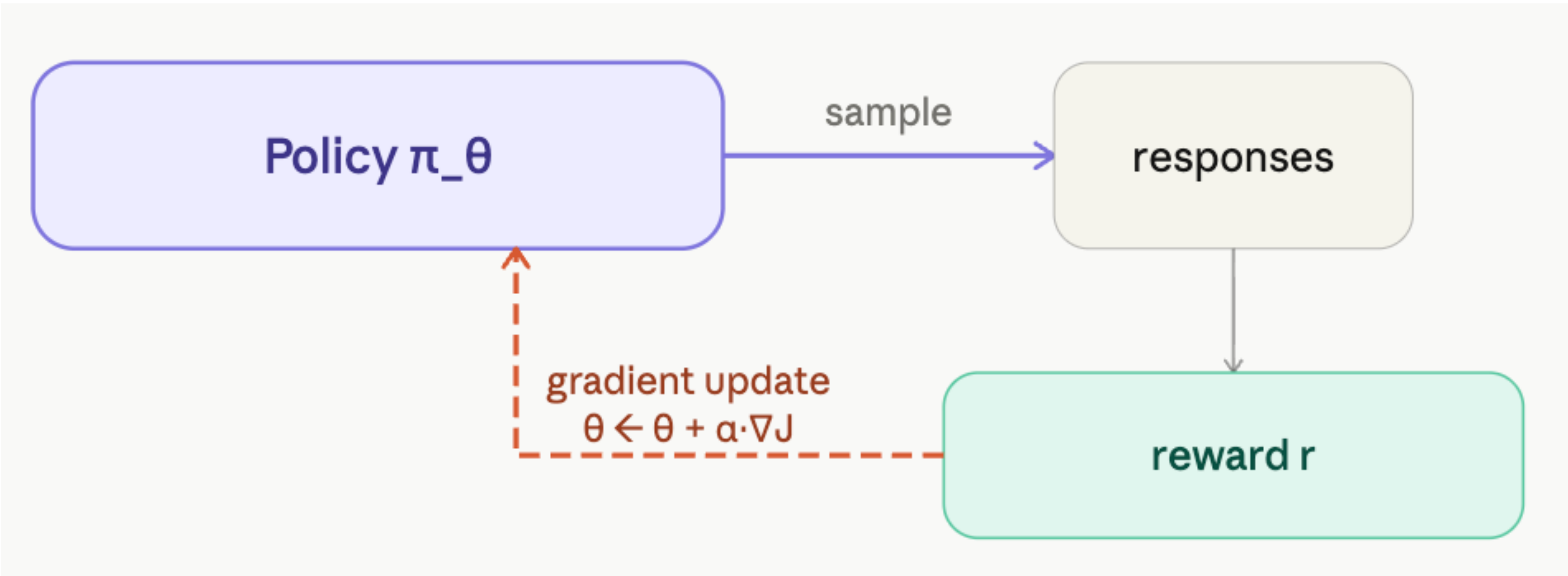
- We want to learn to generate outputs $\mathbf{y}^{(i)}$ given inputs $\mathbf{x}^{(i)}$
- Use this data to learn a model to score outputs
  - Good outputs → high score, bad outputs → low score
  - This will be our reward model
- Use this model in reinforcement learning to fine-tune your LM

UCLA ENGINEERING
Computer Science

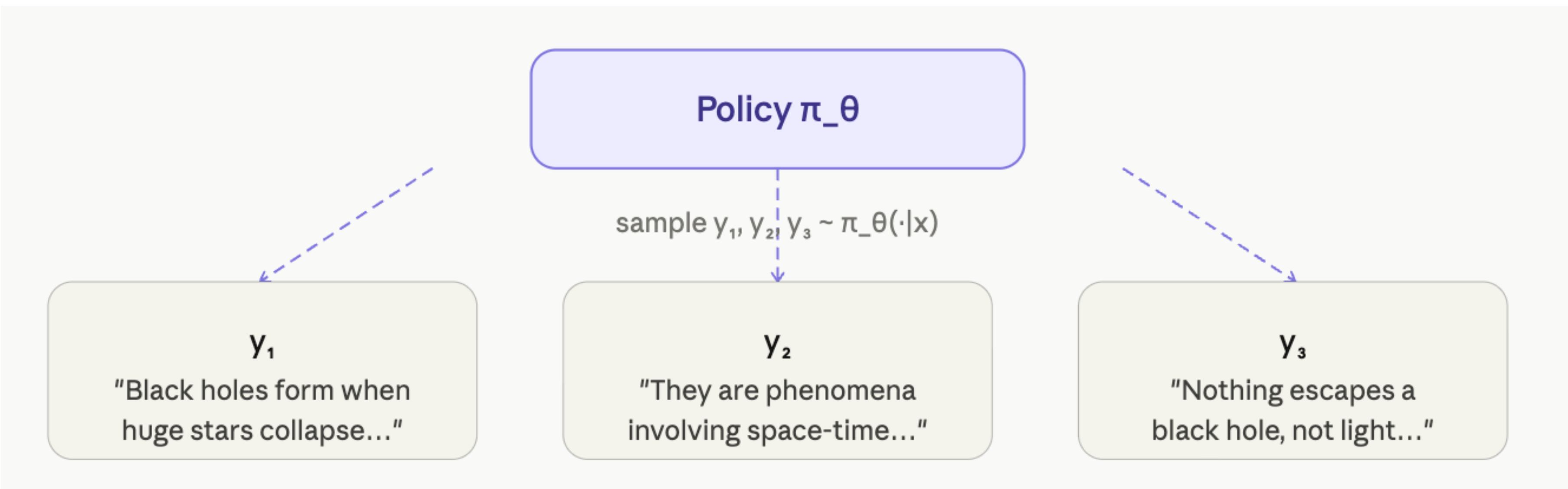
Reinforcement Learning
-- A brief introduction

- There are various RL methods
  - REINFORCE (a simple gradient-based algorithm)
  - Proximal policy optimization (PPO)
  - Direct Preference Optimization (DPO)

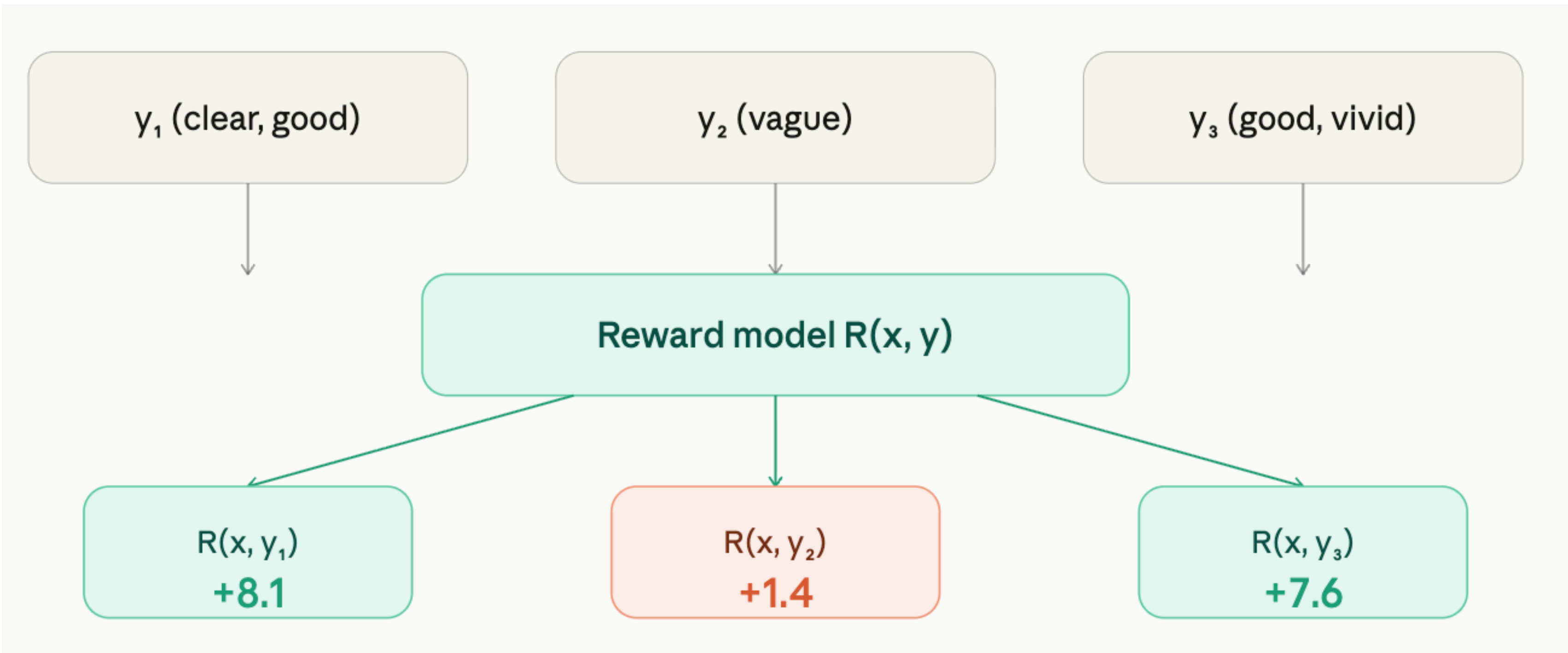
General Idea



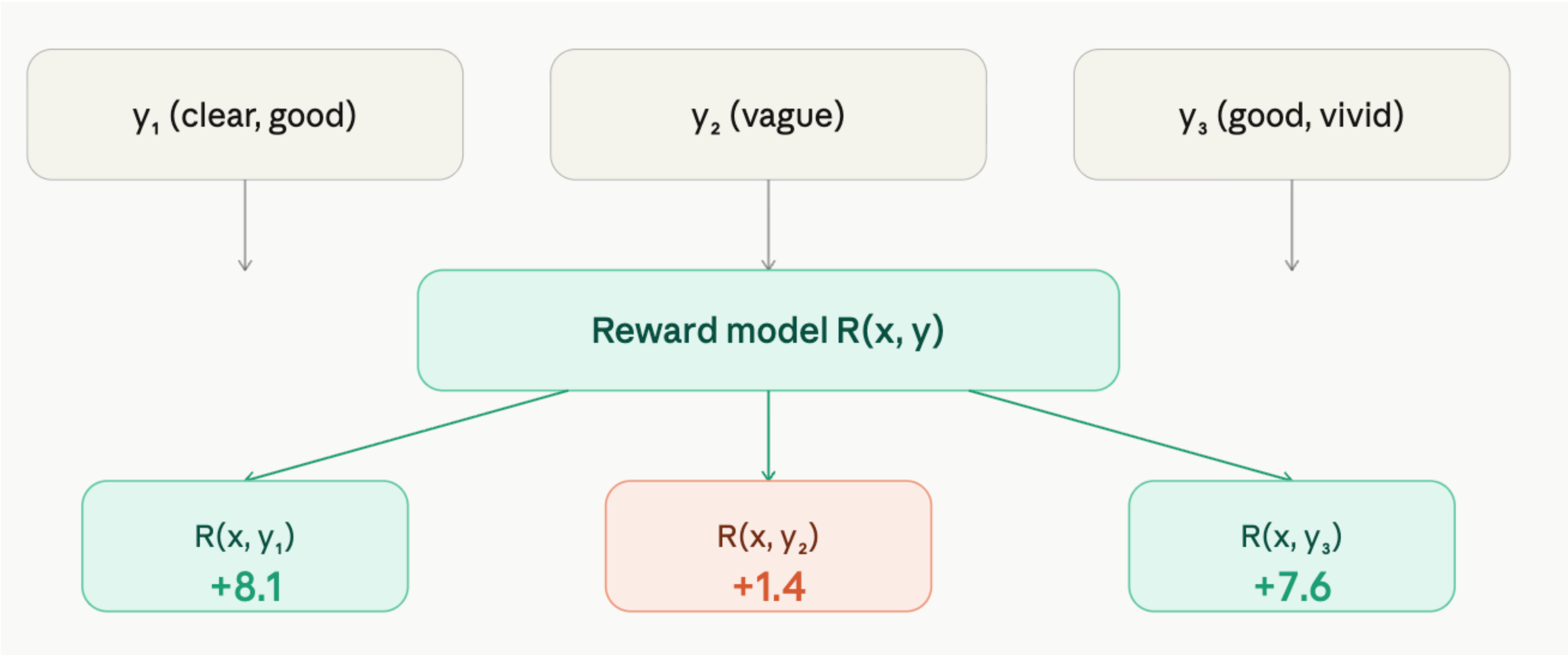
General Idea



General Idea



General Idea



General Idea

$$\nabla\_\theta J(\theta) = E[\,(r - b) \cdot \nabla\_\theta \log \pi\_\theta(y|x)\,]$$

Estimate with our 3 samples:

$y_1 \rightarrow A_1 = +2.4$	$y_2 \rightarrow A_2 = -4.3$	$y_3 \rightarrow A_3 = +1.9$
high reward, above baseline	low reward, below baseline	good, slightly above baseline
$+2.4 \cdot \nabla \log \pi(y_1\|x)$	$-4.3 \cdot \nabla \log \pi(y_2\|x)$	$+1.9 \cdot \nabla \log \pi(y_3\|x)$
push y_1 up ↑	push y_2 down ↓	push y_3 up ↑

Presentation

❖ Give 1-5 (5=best) rating for the presetnation

Go to

join.iClicker.com

LCKW

UCLA ENGINEERING
Computer Science